TP - Streams: a Matrix Product by Columns, then by Panels

Oguz Kaya
oguz.kaya@universite-paris-saclay.fr

Part 1

Streaming matrix multiplication using CUDA

We compute the product $C = AB$ of two square matrices of size n, stored column-major: the element $X(i, j)$ is at `X[i + j * n]`, so each column occupies n consecutive values in memory. The matrix A is copied to the GPU once, at the start. The matrix B, on the other hand, is sent to the GPU piece by piece, and each piece of C is brought back as soon as it is computed, as one would do with data that does not fit in GPU memory or that arrives over time. The goal of the lab is to overlap these copies with the computation, using streams as in the lecture.

You work in **matmul-streams.cu**. Everything except the functions you write is provided: the data, a reference result computed by cuBLAS on the whole matrices, the CPU matrix product used in version 4, and the timing and the correctness check of each version. As given, the program compiles and runs, and prints `WRONG` for every version that is not written yet. Answer the questions in a comment above each function.

Compile and run with

```
nvcc -O2 -arch=native matmul-streams.cu -o matmul-streams -lcublas -Xcompiler -fopenmp -lgomp
./matmul-streams
```

`-lcublas` links cuBLAS; `-Xcompiler -fopenmp` passes `-fopenmp` to the host compiler, which compiles the CPU code, so that `cpu_dgemm` runs on all the cores; `-lgomp` links the OpenMP runtime.

Part 2

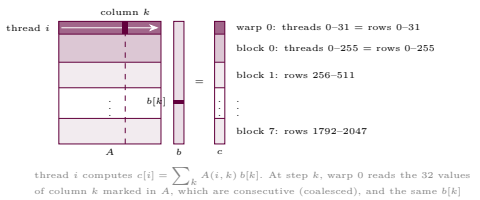
Exercise 1: column by column, in single precision

In this exercise the matrices are in single precision and $n = 2048$. Each column of C is computed on its own, by a matrix-vector product kernel of yours: $C(:, j) = A\,B(:, j)$.

2.1

Version 1: the matrix-vector kernel and the sequential loop

Write the kernel `matvec(n, A, b, c)`, which computes $c = A\,b$ with one thread per row: thread i computes `c[i]` as the sum of `A[i + k * n] * b[k]` over k. Launch it with blocks of `BS` $= 256$ threads, and enough blocks to cover the n rows; a thread past the last row does nothing. Then write `version1`, which processes the columns one after the other: for each column j, it copies $B(:, j)$ into the GPU buffer `dB` with `cudaMemcpy`, runs the kernel, and copies the result from `dC` back into $C(:, j)$. Nothing overlaps in this version; it is the baseline for the next one.



one thread per row of A, blocks of 256 threads (not to scale)



column j: copy in, compute, copy out, then column $j + 1$

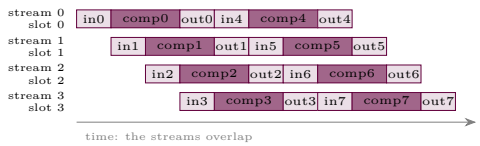
Version 1 – questions

a) Write `matvec` and `version1`, and check that the result is correct.

2.2

Version 2: S streams

Version 2 runs the same loop with S streams, so that the work on different columns overlaps. Since several columns are now in flight at the same time, each stream needs its own buffers on the GPU: `dB` and `dC` hold S columns each, and stream s uses the column at `dB + s * n` (and likewise in `dC`). Column j is handled by stream j mod S: copy it in with `cudaMemcpyAsync`, launch `matvec` in that stream, and copy the result out with `cudaMemcpyAsync`. Create the streams before the loop, wait for all of them with `cudaDeviceSynchronize()` after it, then destroy them. The host arrays B and C are allocated in pinned memory, which asynchronous copies require. `main()` runs this version with $S = 1, 2, 4, 8$ and 16.



column j in stream j mod 4, slot j mod 4; in a stream, in order

Version 2 – questions

a) Write `version2`, and check the result for every S.

b) Which S is the fastest, and how much faster than version 1 is it? Explain how the time varies with S, using the number of blocks of one `matvec` and the number of SMs of the GPU.

c) With $S = 1$ the time is the same as in version 1. Why?

d) Column $j + S$ reuses the GPU buffers of column j. Why are they guaranteed to be free by then, without any extra synchronisation?

e) Set the macro `PINNED` to 0 at the top of the file, so that B and C are allocated with `malloc`, and run again. What happens to the times, and why?

Part 3

Exercise 2: panels, cuBLAS and the CPU, in double precision

In this exercise the matrices are in double precision and $n = 1536$. Instead of single columns, B is now sent in **panels** of M consecutive columns, which occupy $M \cdot n$ consecutive values in memory, and each panel of C is computed by cuBLAS. We use double precision because GPUs such as ours are 32 to 64 times slower in double than in single precision, whereas a CPU core is only twice as slow: this is what makes the CPU a useful partner in version 4.

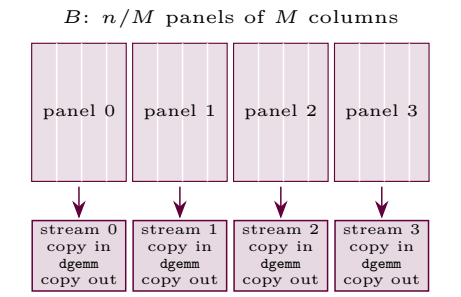
3.1

Version 3: panels with cuBLAS

Version 3 has the structure of version 2, with panels instead of columns and $S = 4$ streams. Panel p goes to stream p mod S, and the GPU buffers of a stream now hold a whole panel: stream s uses `dB + s * M * n` and `dC + s * M * n`. The product of a panel, $C(:, p) = A\,B(:, p)$, is an $n \times M$ matrix product, which you compute with

```
cublasDgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, n, M, n,
            &one, dA, n, dBs, n, &zero, dCs, n)
```

where `dBs` and `dCs` are the stream's buffers. cuBLAS launches its kernels in the stream attached to the handle, so call `cublasSetStream(handle, stream)` before each `cublasDgemm`. `main()` runs this version with $M = 8, 32, 128$ and 512.



one `cudaMemcpyAsync` per panel ($M \cdot n$ doubles), then `cublasDgemm` on the $n \times M$ product

the chunks are now panels: fewer, bigger operations

Version 3 – questions

a) Write `version3`, and check the result for every M.

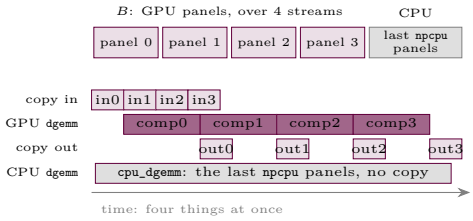
b) Which M is the fastest? Why is $M = 8$ so slow, and why is $M = 512$ slower than 128? (Think about what cannot overlap at the beginning and at the end of the loop.)

c) The reference computes the same product with a single `cublasDgemm` on the whole matrices, between one copy of B in and one copy of C out, without streams. How much faster than the reference is your best panel version?

3.2

Version 4: the CPU joins in

While the GPU computes, the CPU sits idle. In version 4 it computes the last `npcpu` panels of C itself, directly in host memory, so these panels need no copy at all. The provided function `cpu_dgemm(m, n, k, A, lda, B, ldb, C, ldc)` computes $C = AB$ on all the cores of the CPU, for column-major matrices with C of size $m \times n$ and A of size $m \times k$; it is declared at the top of the file and implemented at the end, with AVX2 and OpenMP. `main()` prints its speed on one panel. The GPU handles the other panels exactly as in version 3. Issue all the GPU work first, then call `cpu_dgemm` once on the CPU's panels, then wait for the streams: the copies in, the GPU's products, the copies out and the CPU's product then all run at the same time. `main()` runs this version with panels of $M = 128$ columns and `npcpu` $= 0, 1, 2, 3$ and 4. On the lab machine, set `OMP_NUM_THREADS` to the number of cores.



the GPU work is issued first, then the CPU computes

Version 4 – questions

a) Write `version4`, and check the result for every `npcpu`. Which `npcpu` is the fastest, and how much faster than version 3 is it?

b) Why must the GPU work be issued before the call to `cpu_dgemm`, rather than after it?

c) In single precision the CPU gets only 2 times faster than in double, whereas the GPU gets 32 to 64 times faster. What share of the work would the CPU get then, and what would it gain?